

Algorithm Design Project (Phase 2): Prompt Optimization with Classic Search and Local LLM Evaluation

Algorithm-Project-PromptOptimization-Group11

February 13, 2026

Abstract

We study prompt optimization as a discrete search/optimization problem. A prompt is represented as a structured object that can be modified by explicit edit operators to generate neighboring prompts. Each prompt is evaluated on known tasks (QA and summarization) using reference-based metrics (Exact Match and ROUGE-L) and a local LLM backend (Ollama). We implement and compare multiple classic methods under a fixed evaluation budget: Hill Climbing (local search), BFS (graph search), and Beam Search (bounded search). We additionally implement heuristic pruning, including a novelty filter that can be based on either token overlap or embedding cosine similarity using an Ollama embedding model. We perform empirical evaluation by measuring runtime and peak memory, logging convergence traces, and saving failure cases for qualitative analysis.

1 Problem Definition

1.1 Goal

Given a dataset $D = \{(x_i, y_i)\}_{i=1}^n$, an initial prompt p_0 , a neighbor generator G , an evaluator E , and an evaluation budget B , find a prompt p^* that maximizes a known score:

$$p^* = \arg \max_{p \in \mathcal{P}} m(p),$$

where $\mathcal{P}$ is the set of prompts reachable from p_0 via G (within budget constraints).

1.2 Inputs and Outputs

Inputs:

- Task $T \in \{\text{qa}, \text{summarization}\}$
- Dataset D (at least 10 inputs per task, tagged simple/medium/hard)
- Initial prompt p_0
- Neighbor generator G based on explicit edit operators
- LLM backend M (Ollama for experiments; MockLLM for tests)
- Budget B (maximum number of prompt evaluations)

Outputs:

- Best prompt p^* and its score $m(p^*)$
- Convergence trace files (CSV) for plotting best-so-far score vs. evaluation index
- Failure-case logs (JSON) per run, including input, prediction, reference, and metric score
- Empirical measurements: runtime and peak memory usage

2 Datasets and Metrics

2.1 Tasks

We evaluate on two tasks:

- **Question Answering (QA):** short answers with strict formatting sensitivity.
- **Summarization:** produce concise summaries of multi-sentence passages.

2.2 Metrics

QA: Exact Match (EM). After normalization (case, punctuation, whitespace), EM is 1 if prediction matches the reference, else 0.

Summarization: ROUGE-L. We compute token-level ROUGE-L F1 based on longest common subsequence overlap.

2.3 Scoring Function

We use:

$$m(p) = q(p) - \lambda \cdot \text{cost}(p),$$

where $q(p)$ is the average metric score over the dataset and $\text{cost}(p)$ is a proxy (e.g., prompt length or token usage). In most experiments we set $\lambda = 0$ for pure quality comparison; a cost-aware setting with $\lambda > 0$ is supported by the implementation.

3 Search Space and Prompt Generation

3.1 Prompt Representation

Prompts are structured objects containing:

- task identifier,
- instruction string,
- a list of constraints,
- output format specification.

Rendering from the structured representation to prompt text is deterministic for reproducibility and hashing.

3.2 Neighbor Generation

Neighbors are generated using explicit edit operators, for example:

- add, remove, or reorder constraints,
- change output format requirements,
- toggle optional directives (e.g., brevity, verification).

The effective branching factor is controlled via a cap `max_neighbors`.

4 Algorithms Implemented

All methods share the same evaluation pipeline:

1. generate candidate prompts via neighbor function G ,
2. evaluate candidates using local LLM inference and a reference-based metric,
3. select the best prompt subject to a fixed evaluation budget B .

4.1 Hill Climbing

Hill climbing repeatedly moves to an improving neighbor until no improvement is found or the budget is exhausted.

Algorithm 1 Hill Climbing (budgeted)

```
1: procedure HILLCLIMB( $p_0, E, D, B$ )
2:    $p \leftarrow p_0$ 
3:    $s \leftarrow E(p, D)$ 
4:    $(p^*, s^*) \leftarrow (p, s)$ 
5:   while evals <  $B$  do
6:      $N \leftarrow \text{Neighbors}(p)$ 
7:      $(p', s') \leftarrow \arg \max_{q \in N} E(q, D)$ 
8:     if  $s' \leq s$  then
9:       break
10:    end if
11:     $p \leftarrow p', s \leftarrow s'$ 
12:    if  $s > s^*$  then
13:       $(p^*, s^*) \leftarrow (p, s)$ 
14:    end if
15:  end while
16:  return  $(p^*, s^*)$ 
17: end procedure
```

4.2 BFS on the Prompt Graph

BFS explores prompts level-by-level up to a depth limit L with visited-set deduplication and an evaluation budget. BFS is complete within the explored depth but can be expensive due to exponential growth in the worst case.

4.3 Beam Search

Beam search expands level-by-level but keeps only the top k prompts per depth level. This bounded strategy typically provides a better trade-off between exploration and compute than BFS.

4.4 Heuristic Pruning and Embeddings

We implement heuristic constraints to reduce redundant exploration:

- **Length cap:** discard prompts exceeding a maximum word count.
- **Novelty pruning:** discard candidates too similar to previously evaluated prompts.

Novelty can be computed via:

- **Token overlap (Jaccard similarity)**, or
- **Embedding cosine similarity** using an Ollama embedding model (e.g., `granite-embedding:30m`).

5 Complexity Analysis

Let:

- b be capped branching factor (`max_neighbors`),
- L be depth limit,
- k be beam width,
- n be dataset size,
- C_M be cost of one model generation per example,
- B be prompt-evaluation budget.

Evaluating one prompt costs $O(n \cdot C_M)$.

5.1 Hill Climbing

Worst-case evaluations are $O(B)$, so time is $O(B \cdot n \cdot C_M)$ and space is $O(1)$ (plus logs and caches).

5.2 BFS

Worst-case explored nodes up to depth L is $O(b^L)$, also bounded by B . Time is $O(\min(B, b^L) \cdot n \cdot C_M)$; space is $O(\min(B, b^L))$.

5.3 Beam Search

Beam explores approximately $O(k \cdot b \cdot L)$ candidates, also bounded by B . Time is $O(\min(B, kbL) \cdot n \cdot C_M)$; space is $O(k \cdot b)$.

6 System Diagram (Data Flow)

Figure 1 illustrates the pipeline and the roles of the classic algorithms and the LLM.

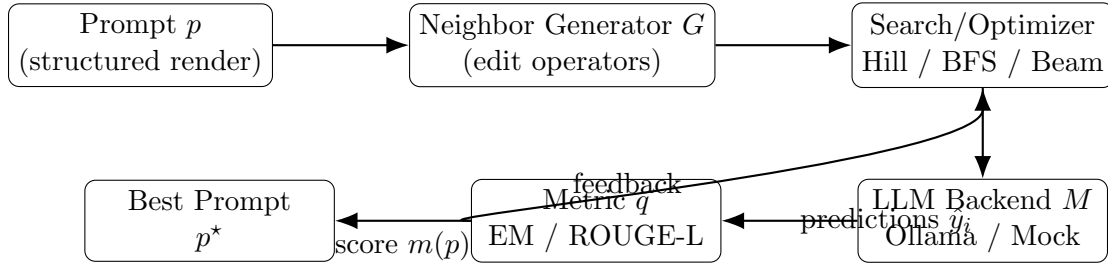


Figure 1: System diagram showing where classic algorithms and the LLM operate, and how data flows.

7 Implementation Overview

7.1 Repository Structure

```

src/
  prompt/ # prompt representation + operators
  evaluation/ # evaluator, metrics, ollama + mock backends
  algorithms/ # hill, bfs, beam (+ heuristics)
  analysis/ # failure cases, embeddings utilities
  utils/ # trace logger, hashing, datasets
scripts/
  run_search.py
  benchmark_phase2.py
  plot_results.py
  make_report_table.py
tests/
results/ # traces, benchmark CSV, plots (ignored by git)
reports/
  
```

7.2 Reproducibility Artifacts

Each benchmark run produces:

- trace CSV: `results/trace_<run_id>.csv`
- failure cases JSON: `results/worst_<run_id>.json`
- summary rows in `results/bench_summary.csv`

8 Empirical Evaluation Protocol

We compare methods under identical settings:

- same dataset and metric,

- same LLM model and temperature (0.0),
- same evaluation budget B and neighbor caps,
- heuristics toggled on/off for controlled analysis.

We measure:

- runtime in seconds via `time.perf_counter()`,
- peak memory via `tracemalloc`,
- convergence via trace logs (best-so-far vs evaluation index).

9 Results

9.1 Benchmark Summary

Table 1 shows the benchmark summary for a small budget $B = 4$ with heuristics enabled.

Task	Method	Budget B	#Evals	Best Score	Runtime (s)	Peak MB
QA	Hill Climbing	4	4	0.9167	63.02	0.199
QA	BFS	4	4	0.9167	61.92	0.153
QA	Beam Search	4	4	0.9167	62.85	0.155
Summarization	Hill Climbing	4	4	0.5259	66.56	0.157
Summarization	BFS	4	4	0.5259	67.13	0.155
Summarization	Beam Search	4	4	0.5259	67.61	0.159

Table 1: Benchmark summary for budget $B = 4$ with heuristics enabled. All methods reached identical best scores under this small evaluation budget.

Under a very small evaluation budget ($B = 4$), all algorithms converge to identical best scores. This indicates limited exploration of the prompt search space; algorithmic differences are expected to become more visible under larger budgets.

9.2 Plots

Figures 2–7 show convergence behavior (best-so-far score vs. evaluation index). To keep figures stacked and in-order, we use the `float` package and the `[H]` placement specifier.

9.2.1 QA Convergence

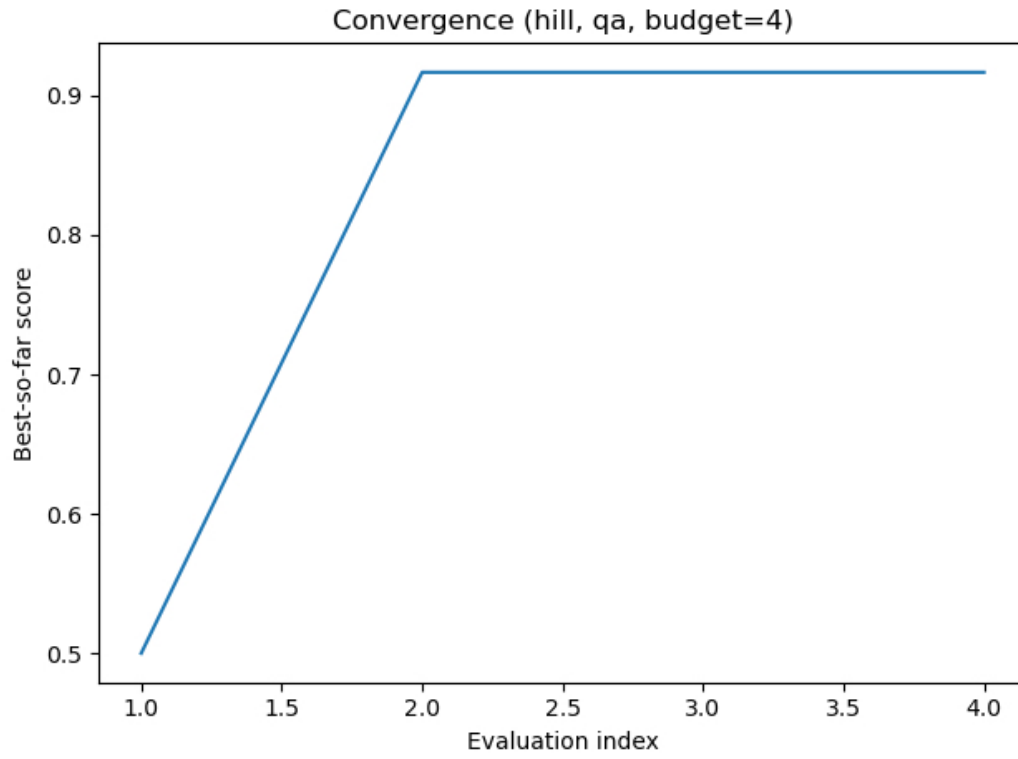


Figure 2: QA convergence under budget $B = 4$ (Hill Climbing).

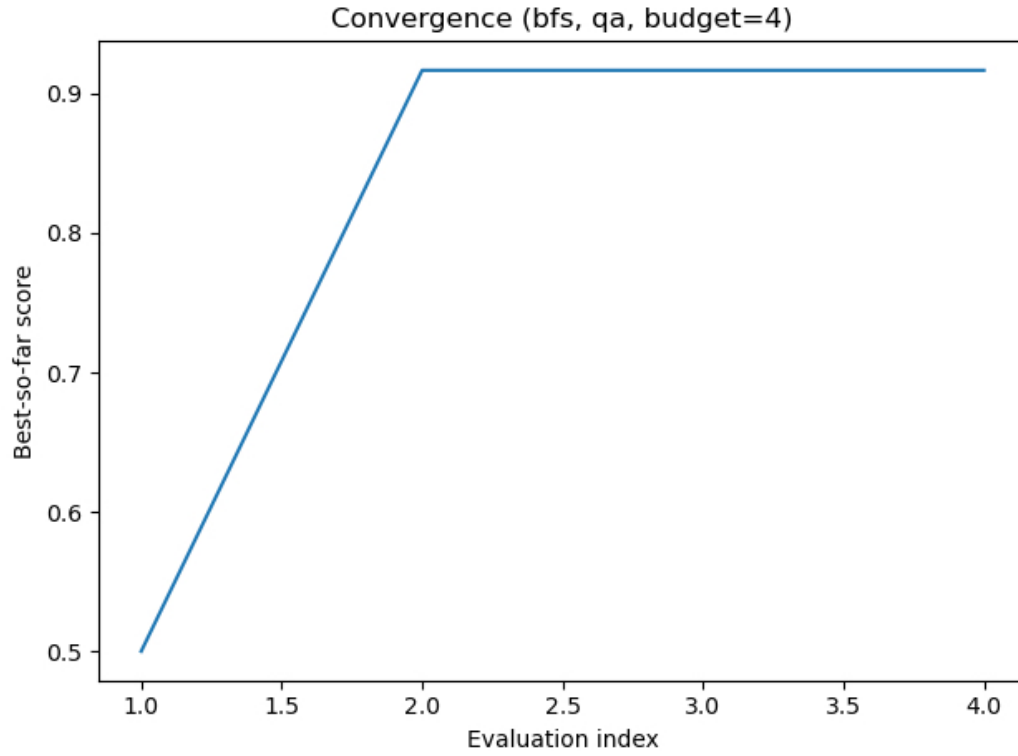


Figure 3: QA convergence under budget $B = 4$ (BFS).

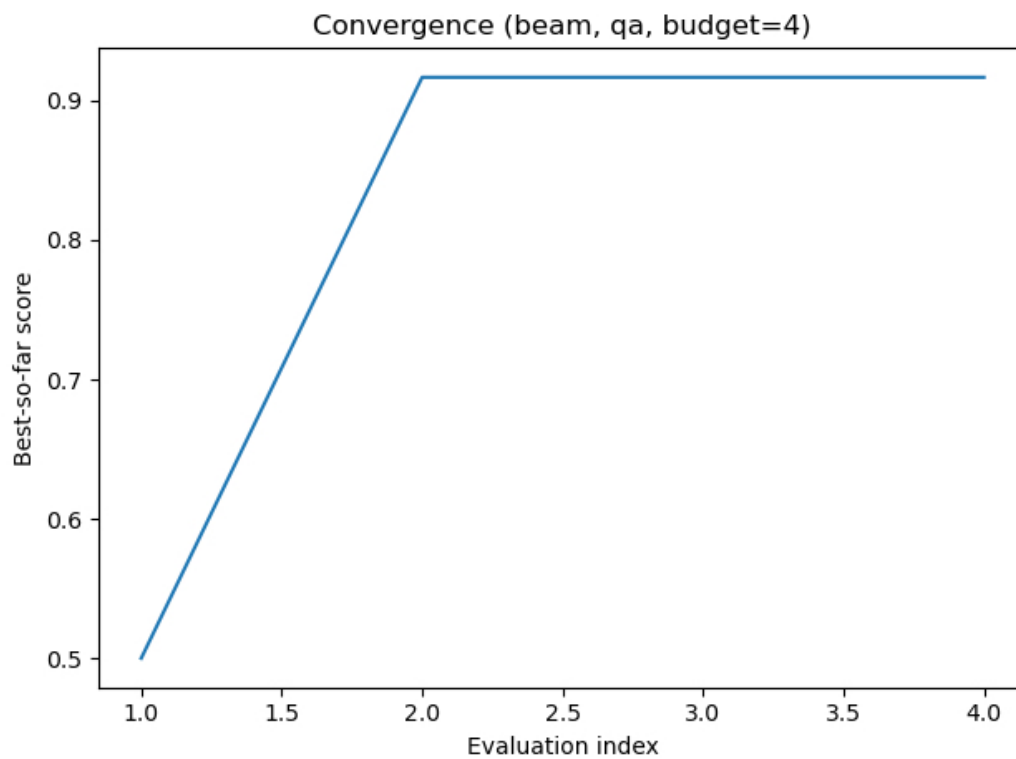


Figure 4: QA convergence under budget $B = 4$ (Beam Search).

9.2.2 Summarization Convergence

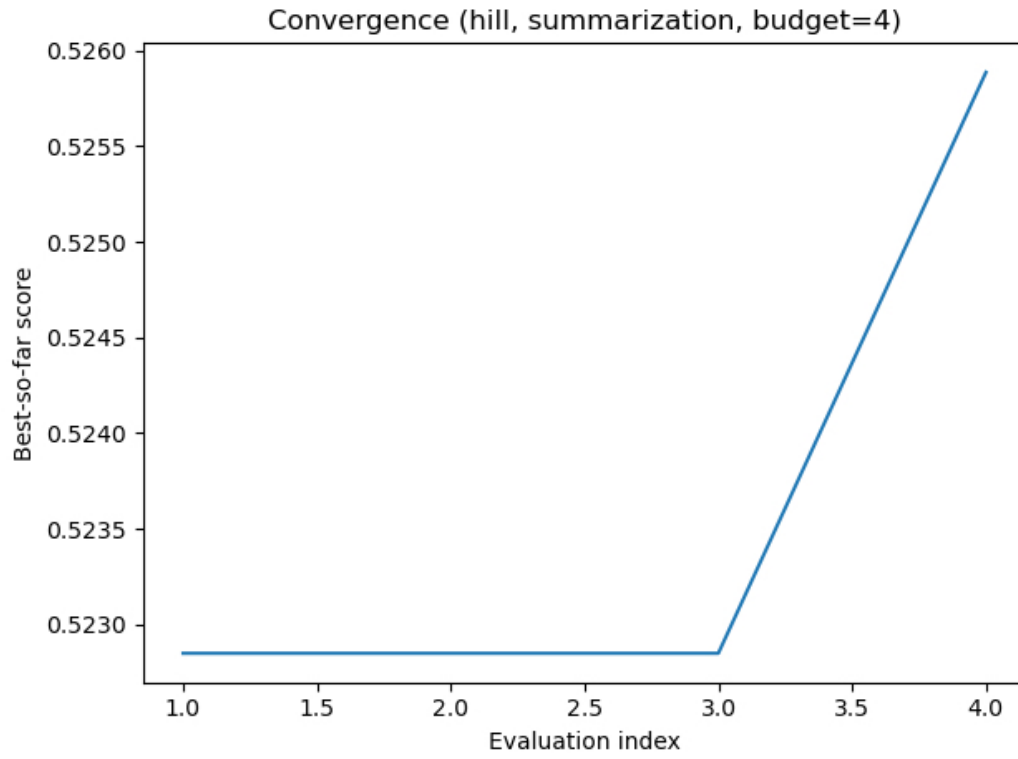


Figure 5: Summarization convergence under budget $B = 4$ (Hill Climbing).

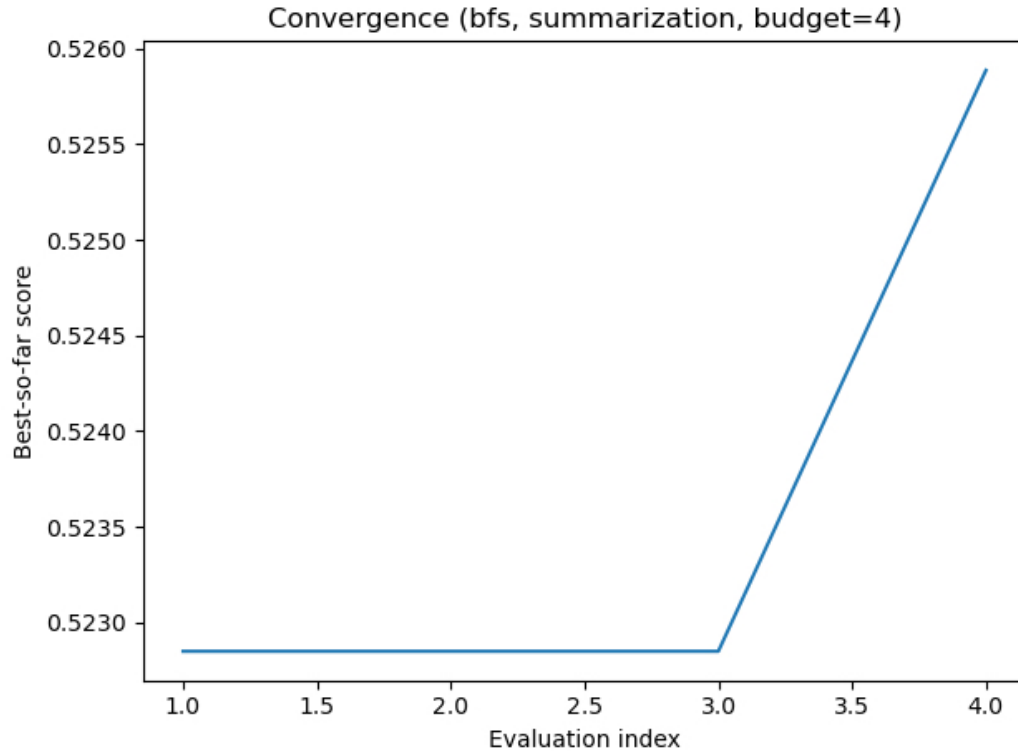


Figure 6: Summarization convergence under budget $B = 4$ (BFS).

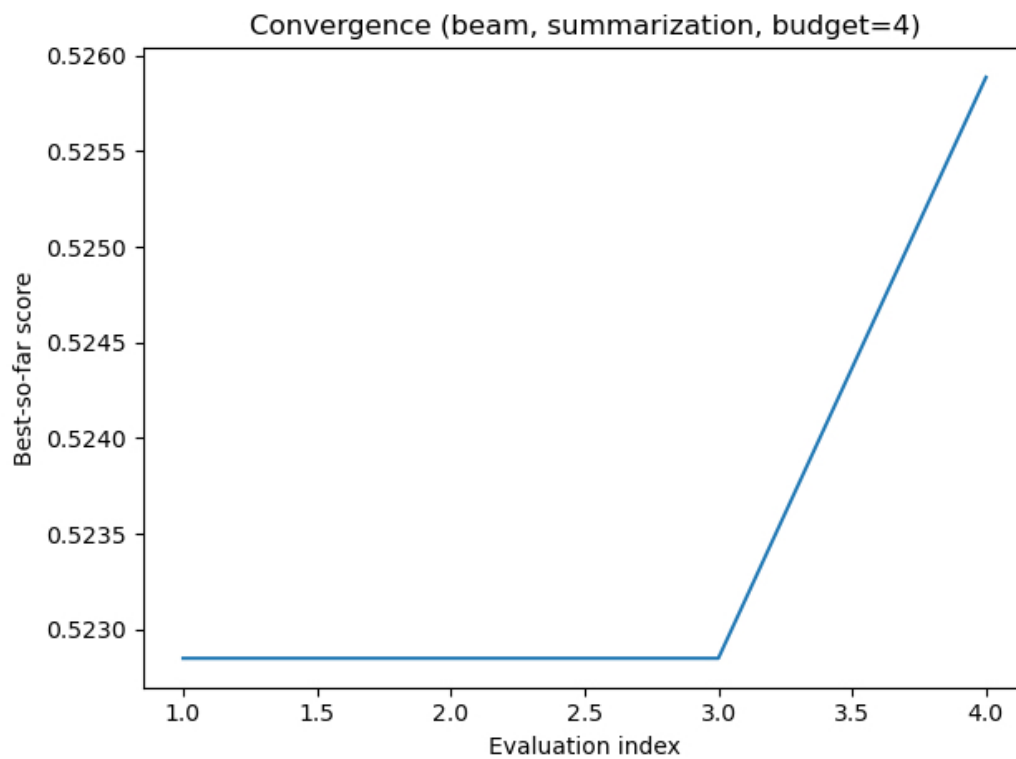


Figure 7: Summarization convergence under budget $B = 4$ (Beam Search).

9.2.3 Runtime and Memory

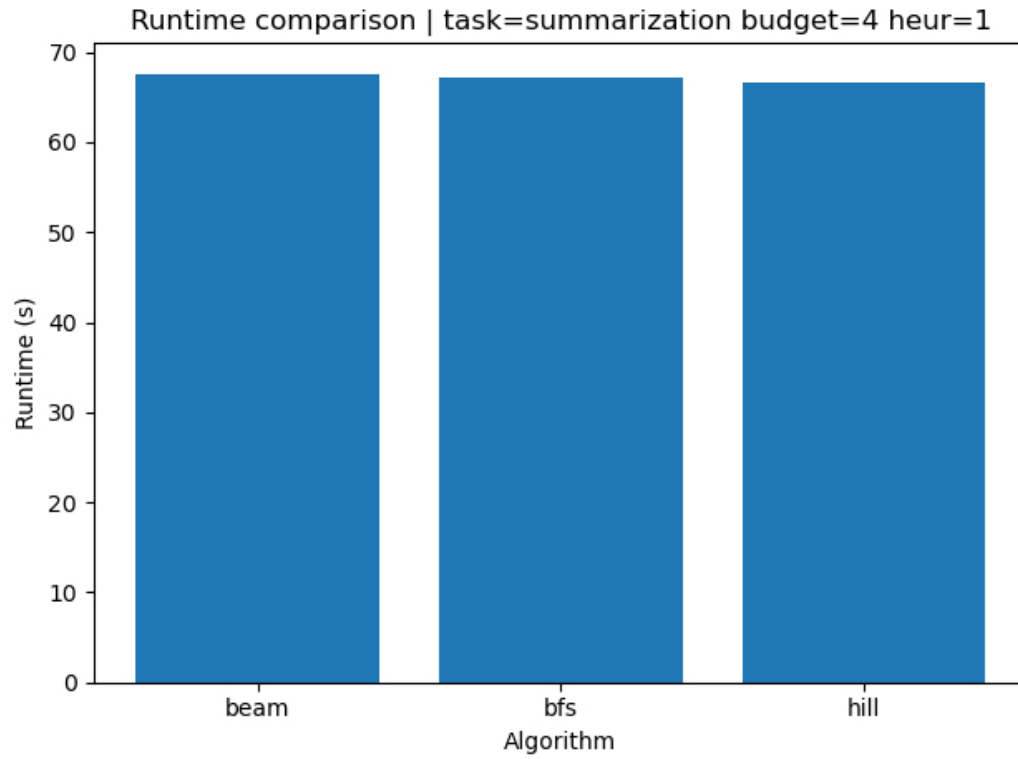


Figure 8: Runtime comparison across algorithms (summarization, $B = 4$, heuristics enabled).

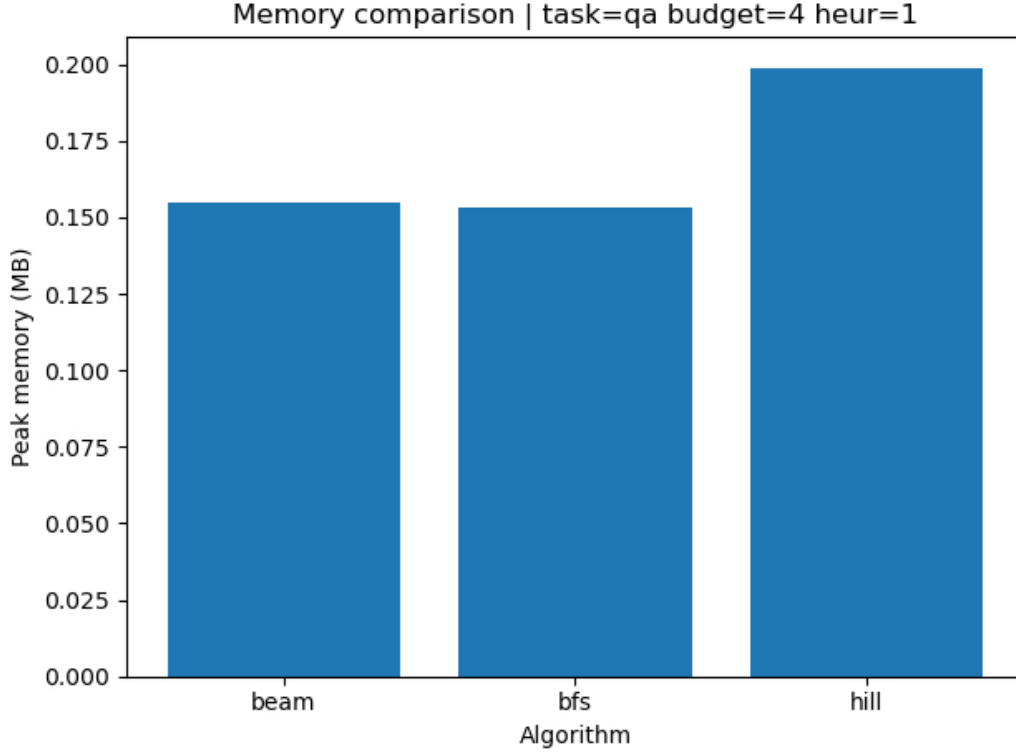


Figure 9: Peak memory comparison across algorithms (QA, $B = 4$, heuristics enabled).

10 Failure Case Analysis

Failure cases are produced automatically and stored as:

```
results/worst_<run_id>.json
```

Typical failure patterns include:

- **QA formatting sensitivity:** answers with extra words may fail exact match.
- **Summarization omissions:** short summaries may miss key entities or numeric details.
- **Model variability:** small local models may vary in adherence to constraints without strongly specified formatting instructions.

11 LLM Role (Explicit)

- **Response generation:** Ollama produces $\hat{y}_i = M(p, x_i)$.
- **Quality evaluation:** metrics compute $q(p)$ deterministically from references.
- **Similarity pruning (optional):** embedding-based novelty uses cosine similarity of prompt embeddings.

12 How to Run (Commands)

12.1 Tests

```
python -m pytest -q
```

12.2 Benchmark and Plots

```
export OLLAMA_MODEL="qwen3:0.6b"
export OLLAMA_EMBED_MODEL="granite-embedding:30m"

python -m scripts.benchmark_phase2 --task both --budgets 4 --max_neighbors 3 --max_depth
      3 --beam_width 4 --heuristics
python -m scripts.plot_results --bench_csv results/bench_summary.csv --out_dir results/
      plots
```

13 Conclusion and Future Work

We implemented and compared multiple classic search and optimization strategies for prompt optimization under a unified evaluation pipeline with local LLM inference (Ollama). We measured runtime and memory, logged convergence traces, and saved failure cases for qualitative analysis. Future improvements include diversity-aware beam search, larger benchmark suites, and multi-objective scoring that jointly optimizes output quality and prompt cost.